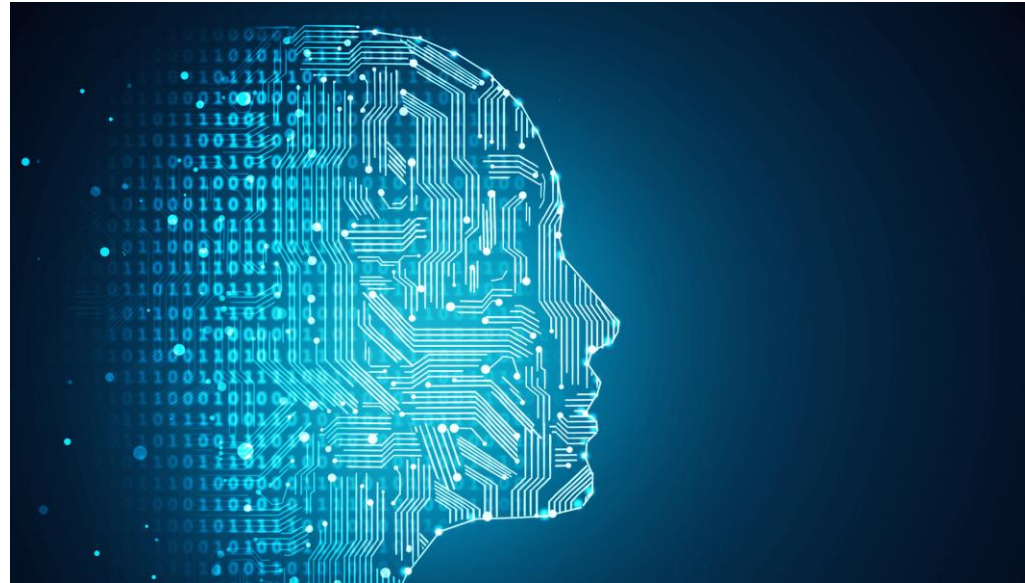


Artificial Intelligence & Machine Learning (Part 2)



Objectifs de ce cours

- Comprendre l'apprentissage non supervisé et le clustering.
- Maîtriser les principaux algorithmes de clustering avec scikit-learn.
- Découvrir les bases du deep learning, du NLP et de l'apprentissage par renforcement.
- Appliquer les connaissances sur un modèle moderne (Cas pratiques / Projets)

Plan

- Module 1 : Clustering – Introduction et K-Means
- Module 2 : Clustering probabiliste et mixed membership (GMM, LDA)
- Module 3 : Clustering hiérarchique, DBSCAN et réduction de dimension (PCA)
- Module 4 : Deep Learning, NLP et Apprentissage par renforcement
- Projet de groupe : présentation d'un modèle moderne

Module 1 : Clustering – Introduction et K-Means

Apprentissage non supervisé : rappel

- Contrairement à l'apprentissage supervisé, les données ne sont pas étiquetées : pas de variable cible (y).
- Le système apprend à partir des données elles-mêmes, sans indication.
- Objectif : découvrir des structures, des groupes ou des régularités cachées dans les données.
- Exemples : segmentation de clients, détection d'anomalies, regroupement de documents.

Clustering : définition

- Le clustering (ou partitionnement) est une technique d'apprentissage non supervisé qui consiste à regrouper des individus similaires dans un même cluster (groupe).
- Deux individus du même cluster doivent être proches, et deux individus de clusters différents doivent être éloignés.
- La notion de « similarité » se traduit mathématiquement par une distance (euclidienne, Manhattan, ...).

Clustering : cas d'usage

- Marketing : segmentation de la clientèle (profils d'achat).
- Médecine : regroupement de patients aux symptômes similaires.
- Image : compression, regroupement de pixels par couleur.
- Texte : regroupement automatique d'articles ou de tweets par sujet.
- Détection d'anomalies : points isolés qui n'appartiennent à aucun cluster.

Les grandes familles de clustering (1/2)

- Partitionnement (K-Means) : on fixe le nombre K de clusters à l'avance.
- Hiérarchique : on construit un arbre (dendrogramme) de regroupements successifs.
- Basé densité (DBSCAN) : un cluster est une zone dense de points. Détecte aussi le bruit (outliers).
- Dynamique (Mean-Shift, Affinity Propagation) : le nombre de clusters n'est pas fixé à l'avance.

Les grandes familles de clustering (2/2)

- Probabiliste / soft clustering (Gaussian Mixture Models) : chaque point a une probabilité d'appartenir à chaque cluster.
- Mixed membership (LDA) : un individu peut appartenir à plusieurs clusters à la fois, dans des proportions différentes. Exemple : un document qui parle à 60 % de sport et à 40 % de politique.
- Info : il n'existe pas d'algorithme « meilleur » dans l'absolu. Le choix dépend des données et de l'objectif.

K-Means : principe

- K-Means est l'algorithme de clustering le plus utilisé.
- Idée : partitionner les données en K clusters, chaque cluster étant représenté par son centre (centroïde).
- Chaque point est associé au cluster dont le centroïde est le plus proche.
- L'algorithme cherche à minimiser la somme des distances au carré entre chaque point et son centroïde (inertie).

K-Means : algorithme

- 1. Choisir le nombre K de clusters.
- 2. Initialiser aléatoirement K centroïdes.
- 3. Assigner chaque point au centroïde le plus proche.
- 4. Recalculer la position de chaque centroïde comme la moyenne des points de son cluster.
- 5. Répéter 3-4 jusqu'à ce que les centroïdes ne bougent plus (convergence).

K-Means : comment choisir K ?

- Méthode du coude (elbow method) : tracer l'inertie en fonction de K et repérer le « coude ».
- Score de silhouette : mesure à quel point un point est bien dans son cluster (valeur entre -1 et 1, plus proche de 1 = meilleur).
- Info : K-Means suppose des clusters sphériques de taille similaire. Il est sensible à l'initialisation (utiliser k-means++).

K-Means : comment choisir K ?

La méthode du coude (elbow method)

L'idée : essayer plusieurs valeurs de K et regarder comment évolue la qualité du clustering.

Mesure utilisée : l'inertieL'inertie, c'est la somme des distances au carré entre chaque point et le centroïde de son cluster.

En formule simple :

- Inertie = $\sum (\text{distance entre un point et son centroïde})^2$

K-Means : comment choisir K ?

Plus l'inertie est faible, plus les points sont proches de leur centroïde, donc plus les clusters sont compacts.

Comportement de l'inertie quand K augmente

- Si $K=1$, tous les points sont dans un seul cluster $\rightarrow$ inertie très grande.
- Si $K=N$ (autant de clusters que de points), chaque point EST son propre centroïde $\rightarrow$ inertie = 0.
- Entre les deux, l'inertie diminue toujours quand K augmente.

K-Means : comment choisir K ?

L'astuce du "coude"

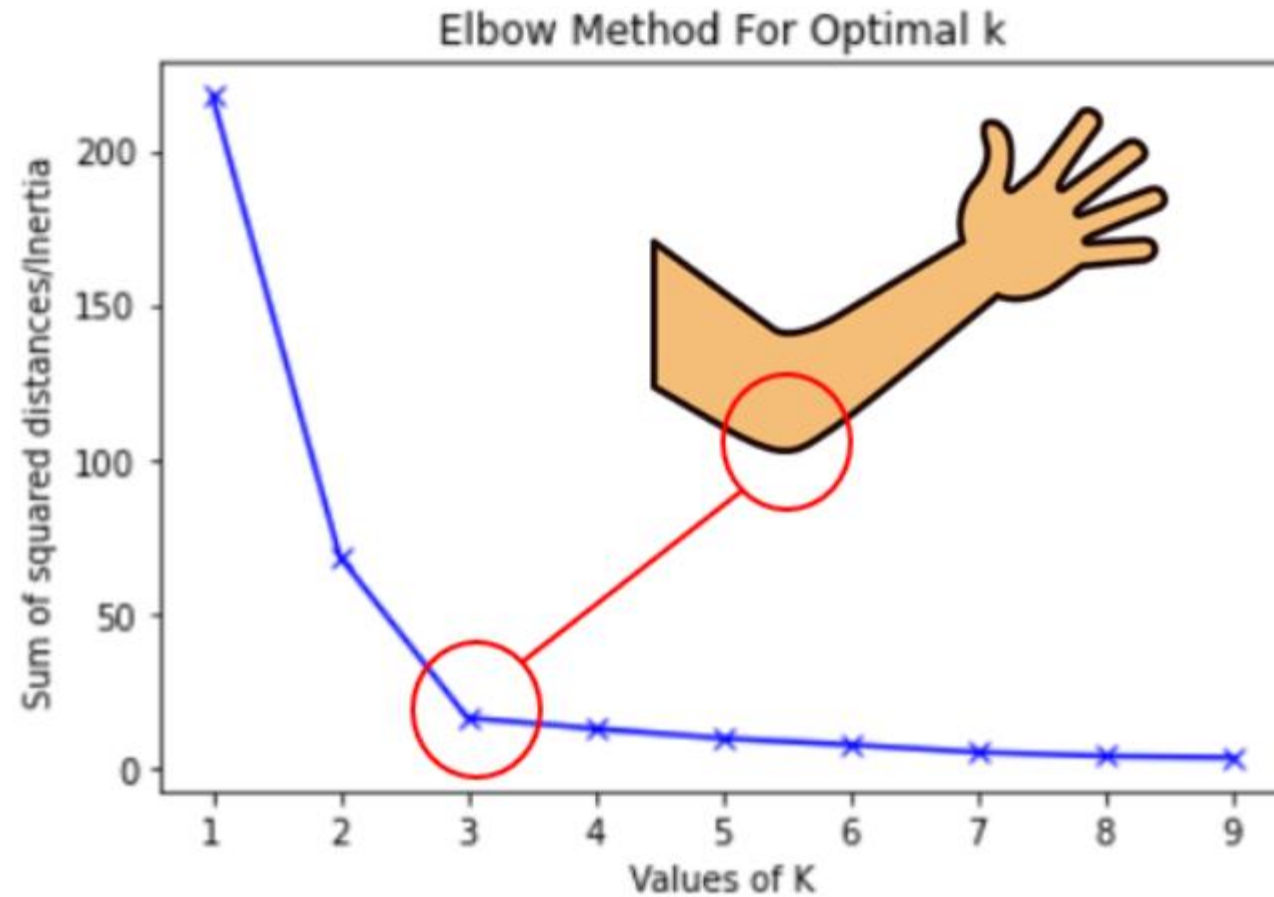
Si on trace la courbe de l'inertie en fonction de K, on voit que :

- Au début, augmenter K fait chuter rapidement l'inertie (on gagne beaucoup).
- À un moment, la courbe se "plie" et continue à descendre mais beaucoup plus lentement (on ne gagne presque plus rien).

Ce point de pliure, c'est le coude. C'est là qu'on choisit K : ajouter des clusters au-delà ne sert plus à grand-chose.

Visuellement : la courbe ressemble à un bras plié. Le coude = le K optimal.

K-Means : comment choisir K ?



Line plot between K and inertia

K-Means : comment choisir K ?

Le score de silhouette :

L'idée : pour chaque point, mesurer à quel point il est "bien" dans son cluster.

Pour un point donné, on calcule :

- a = distance moyenne entre ce point et les autres points de son propre cluster (cohésion interne).
- b = distance moyenne entre ce point et les points du cluster voisin le plus proche (séparation externe).

La silhouette de ce point vaut : $(b - a) / \max(a, b)$

K-Means : comment choisir K ?

Interprétation :

- Valeur proche de 1 $\rightarrow$ le point est bien plus proche de son cluster que des autres. Excellent.
- Valeur proche de 0 $\rightarrow$ le point est à la frontière entre deux clusters. Ambigu.
- Valeur proche de -1 $\rightarrow$ le point est plus proche d'un autre cluster que du sien. Mal classé.

Le **score de silhouette global** est la moyenne des silhouettes de tous les points. On teste plusieurs valeurs de K et on garde celle qui donne le score le plus élevé.

K-Means : code avec scikit-learn

- `from sklearn.cluster import KMeans`
- `model = KMeans(n_clusters=3, init='k-means++', random_state=0)`
- `model.fit(X)`
- `labels = model.predict(X)`
- `centroids = model.cluster_centers_`
- `inertie = model.inertia_`

Exercice

Avec le dataset Iris (sans utiliser les labels) :

1. Appliquer K-Means avec $K=3$ et visualiser les clusters en 2D.
2. Utiliser la méthode du coude pour retrouver la valeur optimale de K .
3. Comparer les clusters obtenus aux vrais labels et calculer le score de silhouette.
4. Refaire le même travail sur le dataset Mall Customers et le dataset des cartes de crédit

Module 2 : Clustering probabiliste et mixed membership

Au-delà de K-Means

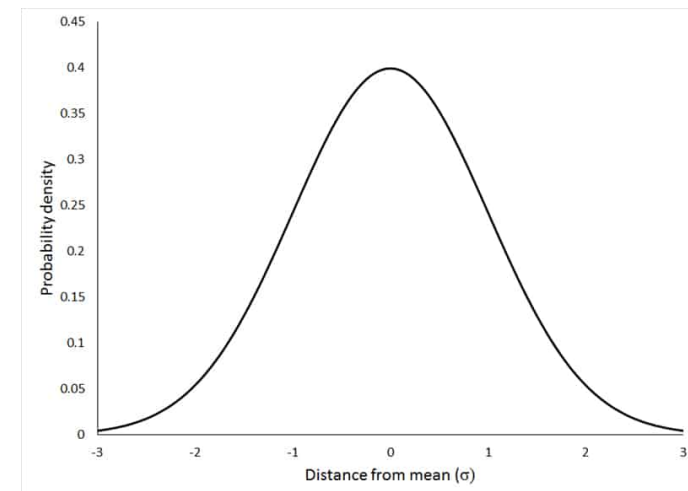
- K-Means attribue chaque point à un seul cluster (hard clustering).
- Mais en pratique, cette frontière est parfois floue : un point peut « ressembler » à plusieurs clusters.
- Le soft clustering associe à chaque point une **probabilité** d'appartenance à chaque cluster.
- Encore plus loin, le mixed membership considère qu'un individu est un mélange de plusieurs clusters (appartenance **réelle** à plusieurs clusters).

Gaussian Mixture Models (GMM) : principe

- Un GMM suppose que les données sont générées par un mélange de K distributions gaussiennes (lois normales).

1. Une loi normale (rappel)

- Une **loi normale** (ou distribution gaussienne) est cette la courbe en cloche, qui décrit une variable qui se concentre autour d'une valeur moyenne, avec une dispersion autour.



Gaussian Mixture Models (GMM) : principe

- Exemple : la taille des adultes français. La plupart font autour de 1m70, quelques-uns sont très petits ou très grands, et la courbe forme une cloche.

Elle est définie par deux paramètres :

- μ (mu) : la moyenne, le centre de la cloche
- σ (sigma) : l'écart-type, la largeur de la cloche

Gaussian Mixture Models (GMM) : principe

2. Une distribution génère des données :

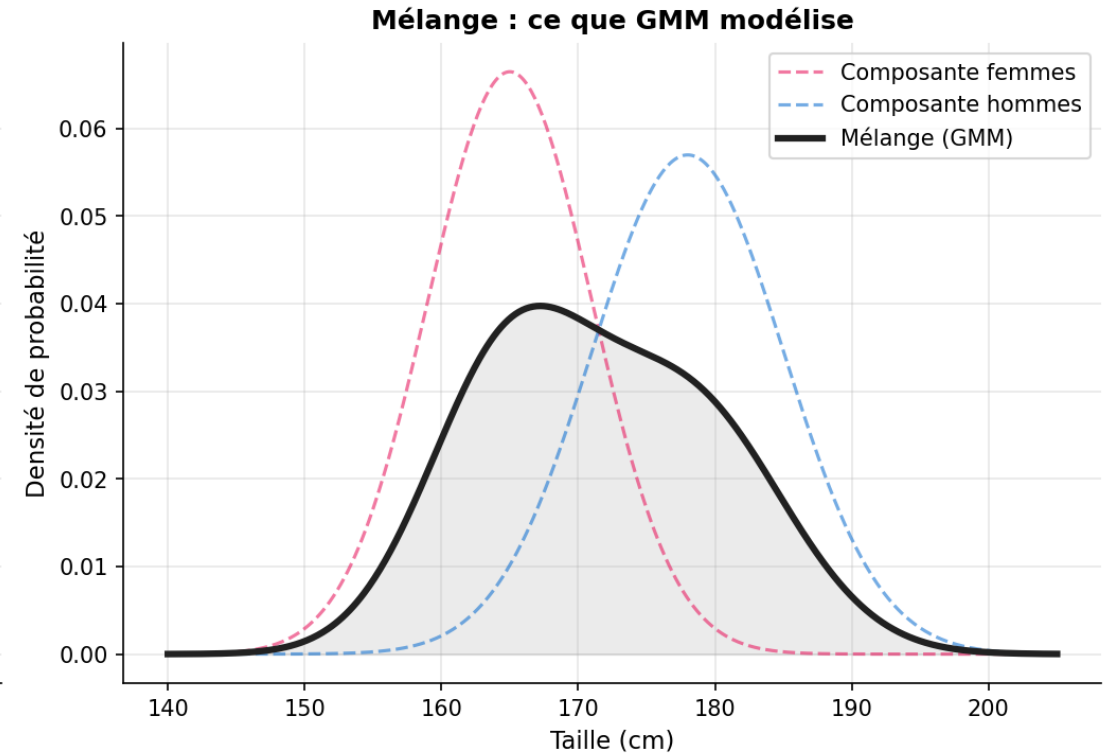
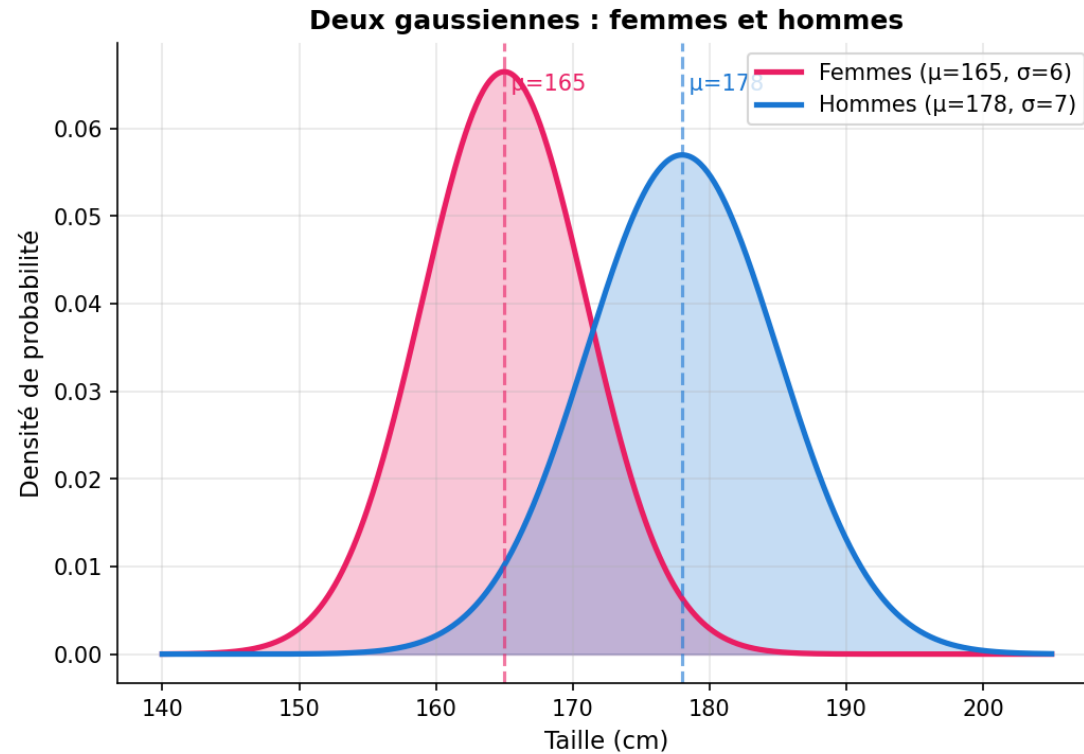
- Quand on dit qu'une distribution **génère** des données, on imagine un "tirage" aléatoire selon cette loi.
- Si on tire 1000 nombres au hasard depuis une loi normale de moyenne 170 et d'écart-type 10, on obtient 1000 valeurs concentrées autour de 170, avec une cloche.
- Autrement dit : la nature construit les données en piochant dans cette distribution.

Gaussian Mixture Models (GMM) : principe

3. Un mélange de K gaussiennes :

- Maintenant, on imagine la mesure de taille de 1000 personnes, mais en mélangeant **femmes** et **hommes**.
- Les femmes ont une moyenne de 1m65, écart-type 6 cm → 1ère gaussienne
- Les hommes ont une moyenne de 1m78, écart-type 7 cm → 2ème gaussienne
- Si on trace l'histogramme de toutes les tailles, on voit deux bosses superposées :

Gaussian Mixture Models (GMM) : principe



Les données sont **un mélange (mixture) de deux gaussiennes**.
Chaque point vient de l'une ou de l'autre, mais on ne sait pas laquelle.

Gaussian Mixture Models (GMM) : principe

4. L'hypothèse de GMM

- **GMM = Gaussian Mixture Model = Modèle de mélange gaussien**

L'algorithme suppose que :

- Les données que je vois sont issues de K gaussiennes mélangées.
Chaque point a été tiré depuis l'une de ces K gaussiennes, mais je ne sais pas laquelle.
- Le travail du modèle GMM: **retrouver les K gaussiennes** (leur centre, leur largeur, leur poids) qui expliquent le mieux les données observées.

Gaussian Mixture Models (GMM) : principe

5. Le lien avec le clustering :

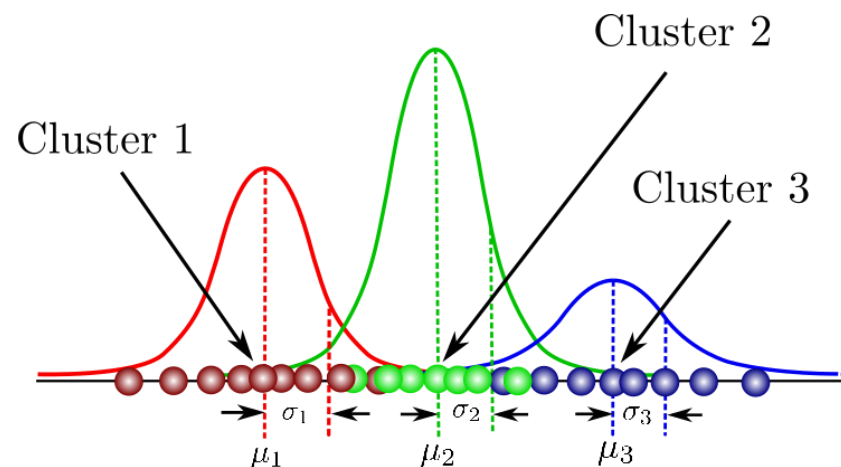
Une fois les K gaussiennes retrouvées :

- Chaque gaussienne = **un cluster**
- Pour chaque point, on peut calculer la probabilité qu'il vienne de chacune des gaussiennes
- On l'assigne au cluster le plus probable (ou on garde les probabilités, c'est ça le "**soft clustering**")

Gaussian Mixture Models (GMM) : principe

Comparaison avec K-Means :

	K-Means	GMM
Hypothèse	Les clusters sont des sphères autour de centroïdes	Les clusters sont des gaussiennes (cloches)
Forme des clusters	Sphérique (ronde)	Elliptique (allongée possible)
Sortie	Chaque point dans 1 cluster	Probabilité d'appartenance à chaque cluster



GMM : code avec scikit-learn

- `from sklearn.mixture import GaussianMixture`
- `model = GaussianMixture(n_components=3, random_state=0)`
- `model.fit(X)`
- `labels = model.predict(X)`
- `probas = model.predict_proba(X)` # probabilité d'appartenance à chaque cluster
- Info : on peut choisir le nombre de composantes avec les critères AIC ou BIC.

GMM

- **AIC (Akaike Information Criterion) et BIC (Bayesian Information Criterion)**
- 2 mesures de qualité du modèle qui combinent deux choses :
 - 1- **À quel point le modèle colle aux données** (plus c'est précis, mieux c'est)
 - 2- **Combien le modèle est complexe** (plus on ajoute de composantes, plus on est complexe)
- L'idée : on veut un modèle **précis mais simple**. Si on met 100 gaussiennes pour 200 points, on aura une précision parfaite... mais on aura juste mémorisé les données (overfitting).
- On choisit le K qui **minimise** l'AIC (ou le BIC). Plus la valeur est basse, mieux c'est.

GMM : code avec scikit-learn

AIC et BIC fonctionnent comme une note avec malus :

Score = (à quel point ça colle aux données) – (malus pour la complexité)

Différence AIC vs BIC :

- **AIC** pénalise un peu la complexité
- **BIC** pénalise plus fort la complexité, donc tend à choisir des modèles plus simples (moins de composantes)

En pratique : on calcule les deux et on regarde où ils s'accordent.

Exercice : calcul des scores AIC et BIC

GMM : Exercice seeds dataset

210 grains de blé répartis en 3 variétés (Kama, Rosa, Canadian), 70 par variété.

7 caractéristiques mesurées par analyse aux rayons X :

Variable	Description
area	Surface du grain
perimeter	Périmètre
compactness	Compacité ($4\pi \cdot \text{area} / \text{perimeter}^2$)
length	Longueur
width	Largeur
asymmetry	Coefficient d'asymétrie
groove	Longueur du sillon
variety	Variété (1=Kama, 2=Rosa, 3=Canadian) — à ignorer pour le clustering

Mixed membership : définition

- Dans le mixed membership, un individu n'appartient pas à un seul cluster, mais est un mélange de plusieurs clusters.
- Exemple : un document peut parler à 60 % de sport, 30 % de politique et 10 % de santé.
- Le modèle le plus connu de mixed membership est la Latent Dirichlet Allocation (LDA), utilisée pour le **topic modeling**.
- Différence avec GMM : GMM donne des probabilités d'appartenance à un cluster (**un seul choix réel**). LDA décompose chaque individu en **proportions** de thèmes.

Mixed membership : définition

Cas d'usage réels de topic modeling

Le topic modeling est utilisé partout où on a beaucoup de texte :

- **Médias** : classer automatiquement des milliers d'articles
- **Recherche** : explorer un corpus scientifique (PubMed, arXiv) pour trouver les grandes thématiques
- **Réseaux sociaux** : identifier les tendances dans les tweets
- **Analyse de plaintes** : regrouper les retours clients par thème
- **Lecture de contrats** : repérer les clauses récurrentes

Mixed membership : définition

Autres utilisations du mixed membership

1. Génétique des populations

- **Question** : un individu descend-il de plusieurs populations ancestrales ?
Application : un humain peut avoir 60% d'ascendance européenne, 30% d'ascendance nord-africaine, 10% d'ascendance d'Asie de l'Ouest. Les tests ADN type AncestryDNA utilisent exactement ce principe.

2. Analyse d'images (pixels mixtes)

- **Question** : un pixel d'image satellite contient quels types de surface ?
Application : un pixel de 100m × 100m vu depuis un satellite peut contenir 50% de forêt, 30% de prairie, 20% de zone urbaine. On parle d'**unmixing spectral**.

Mixed membership : définition

3. Analyse de comportements utilisateurs

- **Question** : quels sont les centres d'intérêt d'un utilisateur ?
Application : un utilisateur Netflix peut être à 40% "amateur d'action", 35% "fan de comédies romantiques", 25% "passionné de documentaires". Permet de recommander des contenus variés.

4. Biologie cellulaire (single-cell RNA-seq)

- **Question** : une cellule en transition exprime quels types cellulaires ?
Application : une cellule souche en différenciation peut être 70% "cellule souche", 30% "cellule musculaire en formation". Très utilisé en recherche biomédicale.

Mixed membership : définition

5. Réseaux sociaux et communautés

- **Question** : à quelles communautés appartient une personne ?
Application : une personne sur LinkedIn peut être à la fois dans la communauté "data science" (60%), "startups" (25%) et "consulting" (15%). Permet d'analyser des réseaux complexes où les frontières communautaires ne sont pas nettes.

6. Analyse de musique

- **Question** : quels genres compose un morceau ? **Application** : un morceau peut être 50% rock, 30% jazz, 20% électronique. Spotify utilise ce type d'analyse pour ses playlists "ambiance".

Mixed membership : définition

Les modèles connus de mixed membership

1. LDA (Latent Dirichlet Allocation)

- Le modèle de référence, créé en 2003 par Blei, Ng et Jordan. Initialement pour le topic modeling, mais il est devenu le cadre général du mixed membership.

2. NMF (Non-negative Matrix Factorization)

- Décompose une matrice en deux matrices plus petites, à valeurs positives. Très utilisé pour le topic modeling (alternative à LDA, souvent plus rapide) et la recommandation. Plus simple mathématiquement que LDA.

Mixed membership : définition

3. PLSA (Probabilistic Latent Semantic Analysis)

- L'**ancêtre de LDA**, créé en 1999. Aujourd'hui rarement utilisé, mais important historiquement. LDA est une version améliorée de PLSA avec une approche bayésienne.

4. STRUCTURE / ADMIXTURE

- Modèles dédiés à la **génétique des populations**, utilisés massivement en biologie. ADMIXTURE est l'outil standard pour estimer l'ascendance à partir d'ADN.

5. Mixed Membership Stochastic Blockmodel (MMSB)

- Pour l'analyse de **réseaux sociaux**. Modélise comment un nœud (personne, page web) peut appartenir à plusieurs communautés simultanément. Utilisé en sociologie computationnelle.

Mixed membership : définition

6. CTM (Correlated Topic Model)

- Extension de LDA qui **modélise la corrélation entre topics**. LDA suppose que les topics sont indépendants ; CTM permet de capter que "Sport" et "Santé" sont souvent corrélés, par exemple.

7. HDP (Hierarchical Dirichlet Process)

- Extension de LDA où le **nombre de topics n'est pas fixé à l'avance** : l'algorithme le découvre automatiquement. Plus puissant mais plus complexe.

Latent Dirichlet Allocation (LDA) : principe

- LDA est un modèle probabiliste utilisé principalement pour le topic modeling (découverte de thèmes dans un corpus de textes).

Hypothèses :

- Chaque document est un mélange de K sujets (topics).
- Chaque sujet est une distribution de probabilités sur les mots du vocabulaire.
- Entrée : un corpus de textes. Sortie : K sujets et, pour chaque document, sa répartition entre ces sujets.

LDA : code avec scikit-learn

- `from sklearn.feature_extraction.text import CountVectorizer`
- `from sklearn.decomposition import LatentDirichletAllocation`
- `X = CountVectorizer(max_features=1000, stop_words='english').fit_transform(docs)` #ne garde que les 1000 mots les plus pertinents
- `lda = LatentDirichletAllocation(n_components=5, random_state=0)`
- `lda.fit(X)`
- `topics = lda.transform(X)` # proportion de chaque sujet pour chaque document

LDA : code avec scikit-learn

Dataset : **fetch_20newsgroups**

Collection d'environ **18 846 messages** postés sur 20 forums de discussion Usenet à la fin des années 1990.

Composition :

18 846 documents au total (11 314 en train, 7 532 en test)

20 catégories (forums) équilibrées (600 à 1 000 messages par catégorie)

Texte brut en anglais, longueur très variable (de quelques lignes à plusieurs milliers)

Les 20 catégories regroupées en 6 thèmes

Thème	Forums
Informatique	comp.graphics, comp.os.ms-windows.misc, comp.sys.ibm.pc.hardware, comp.sys.mac.hardware, comp.windows.x
Loisirs	rec.autos, rec.motorcycles, rec.sport.baseball, rec.sport.hockey
Sciences	sci.crypt, sci.electronics, sci.med, sci.space
Politique	talk.politics.guns, talk.politics.mideast, talk.politics.misc
Religion	talk.religion.misc, alt.atheism, soc.religion.christian
Divers	misc.forsale

NMF (Non-negative Matrix Factorization)

Définition

- NMF (Factorisation de matrices non négatives) décompose une matrice de données en deux matrices plus petites contenant uniquement des **valeurs positives ou nulles**. C'est une alternative à LDA pour découvrir des thèmes ou composantes cachées.

Principe

- À partir d'une matrice document $\times$ mots de taille (n, m) , NMF produit deux matrices :

W $(n \times K)$: composition de chaque document en K topics

H $(K \times m)$: composition de chaque topic en mots

Avec la contrainte que toutes les valeurs soient ≥ 0 .

- L'intuition : on suppose que chaque document est une **somme pondérée positive** de topics, et que chaque topic est une **somme pondérée positive** de mots. La non-négativité rend le résultat naturellement interprétable.

NMF (Non-negative Matrix Factorization)

Utilisations principales :

- **Topic modeling** sur textes
- **Recommandation** : décomposer une matrice utilisateur $\times$ film en facteurs latents
- **Traitement d'images** : décomposer un visage en parties (yeux, nez, bouche)
- **Génomique** : décomposer un profil d'expression génique en signatures

Exercice : Olivetti Faces Dataset (voir reference Lee & Seung (1999))

“Learning the parts of objects by non-negative matrix factorization”

MMSB (Mixed Membership Stochastic Blockmodel)

Définition

- MMSB est un modèle dédié à l'**analyse de réseaux**. Il généralise le mixed membership aux **graphes** : au lieu de dire qu'un nœud (personne, page web, gène) appartient à une seule communauté, il considère qu'il appartient à **plusieurs communautés simultanément**, dans des proportions différentes.

Principe

- Données d'entrée : un **graphe** (matrice d'adjacence). Par exemple, qui suit qui sur Twitter, qui collabore avec qui, qui cite qui.

MMSB (Mixed Membership Stochastic Blockmodel)

Le modèle apprend pour chaque nœud :

- **Sa composition** : quelles communautés il "habite" et dans quelles proportions
- **Sa probabilité d'interagir** avec d'autres nœuds, selon les communautés qu'ils partagent

Cas d'usage typique

Un utilisateur LinkedIn peut être :

- 60% dans la communauté "Data Science"
- 25% dans la communauté "Startups"
- 15% dans la communauté "Consulting"

Exercice : Karate Club Network (Zachary) – `NetworkX\karate_club_graph()`

Info : (Airoldi, Blei, Fienberg, Xing 2008)

HDP (Hierarchical Dirichlet Process)

Définition

- HDP est une **extension de LDA** où le **nombre de topics n'est pas fixé à l'avance**. L'algorithme le découvre automatiquement à partir des données.

Principe

- Avec LDA, on doit choisir K (le nombre de topics) et le tester avec AIC/BIC/perplexité. HDP supprime cette contrainte : il commence avec peu de topics, en ajoute si nécessaire pendant l'apprentissage, et converge vers le bon nombre.

HDP (Hierarchical Dirichlet Process)

Le "hierarchical" dans HDP

- Le modèle a une **structure hiérarchique à deux niveaux** :
- **Niveau global** : il existe un "réservoir" de topics partagés par tous les documents
- **Niveau document** : chaque document pioche ses topics dans ce réservoir, avec ses propres proportions

C'est cette hiérarchie qui permet à l'algorithme de découvrir des nouveaux topics si les données le justifient, sans en imposer un nombre fixe.

Info : modèle plus lent

HDP (Hierarchical Dirichlet Process)

Utilisations principales

- **Exploration de gros corpus inconnus** : analyser des millions de tweets sans savoir combien de thèmes existent
- **Corpus évolutifs** : nouvelles dépêches qui apportent des sujets nouveaux
- **Recherche scientifique** : trouver les sous-domaines émergents dans un corpus d'articles
- **Recherche bayésienne** : autre exemple d'application des processus de Dirichlet

Exercice : NIPS Papers (articles scientifiques d'IA)

Exercice

- Partie 1 (GMM) : générer un jeu de données 2D avec `make_blobs` (clusters allongés). Comparer K-Means et GMM en visualisant les résultats.
- Partie 2 (LDA) : à partir d'un corpus de textes (ex. dataset 20 Newsgroups), identifier 5 sujets principaux et afficher les mots les plus représentatifs de chaque sujet.

Module 3 : Hiérarchique, DBSCAN et réduction de dimension

Clustering hiérarchique : principe

- Construit une hiérarchie de clusters sous forme d'arbre (dendrogramme).
- Approche agglomérative (bottom-up) : chaque point commence dans son propre cluster, puis on fusionne les clusters les plus proches itérativement.
- Approche divisive (top-down) : on part d'un seul cluster contenant tous les points et on le divise.
- Critères de liaison : single (minimum), complete (maximum), average, ward.

Clustering hiérarchique : principe

- **Single linkage (lien minimum)** prend la distance entre les **deux points les plus proches** des deux clusters. C'est la distance "optimiste" : il suffit qu'un seul point d'un cluster soit proche d'un seul point de l'autre pour les rapprocher.
- Résultat : tendance à créer des clusters allongés en "chaîne" (effet chaining), car les groupes se relient dès qu'ils ont un voisin commun. Bon pour détecter des formes non sphériques, mauvais quand il y a du bruit (un seul outlier peut "ponter" deux clusters distincts).

Clustering hiérarchique : principe

- **Complete linkage (lien maximum)** prend l'inverse : la distance entre les **deux points les plus éloignés** des deux clusters. Distance "pessimiste" : pour fusionner deux clusters, il faut que *tous* leurs points soient relativement proches.
- Résultat : clusters compacts et de taille équilibrée, mais sensible aux outliers (un point éloigné fait exploser la distance perçue).

Clustering hiérarchique : principe

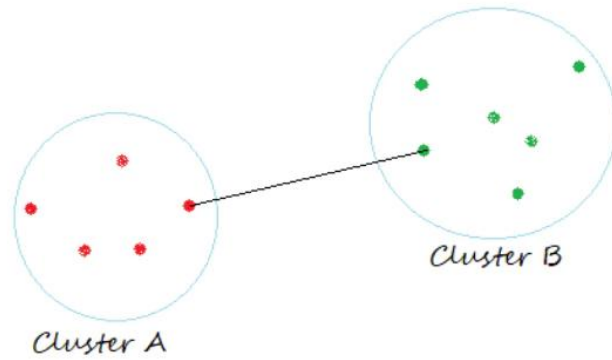
- **Average linkage (lien moyen)** prend la **moyenne des distances** entre toutes les paires de points (un de chaque cluster). C'est un compromis entre single et complete : plus robuste au bruit que single, moins rigide que complete. Donne souvent de bons résultats en pratique sur des données réelles.

Clustering hiérarchique : principe

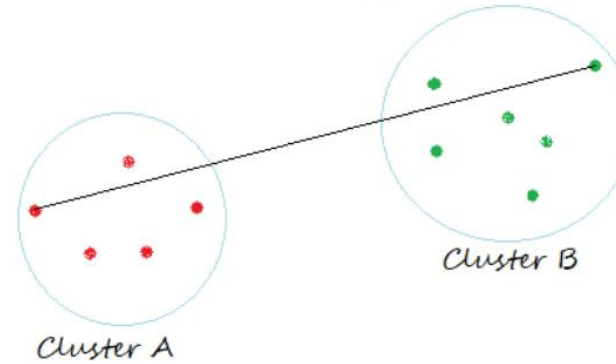
- **Ward linkage** est différent dans sa logique. Il ne mesure pas une distance entre points, mais cherche à **minimiser l'augmentation de la variance intra-cluster** à chaque fusion.
- À chaque étape, il fusionne les deux clusters qui font le moins "exploser" la dispersion totale.
- C'est très proche de la philosophie de K-Means (minimisation de l'inertie), et donne des clusters de tailles similaires et compacts. **C'est le critère par défaut dans scikit-learn**, et celui qui marche le mieux dans la plupart des cas (sauf quand les clusters ont des formes non sphériques).

Clustering hiérarchique : principe

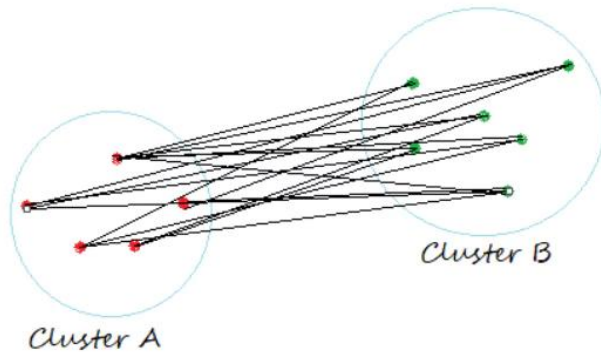
Single Linkage



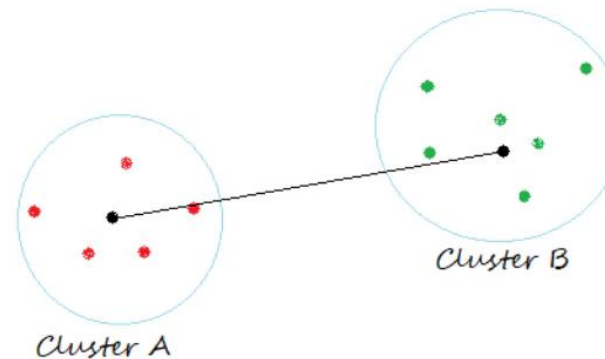
Complete Linkage



Average Linkage



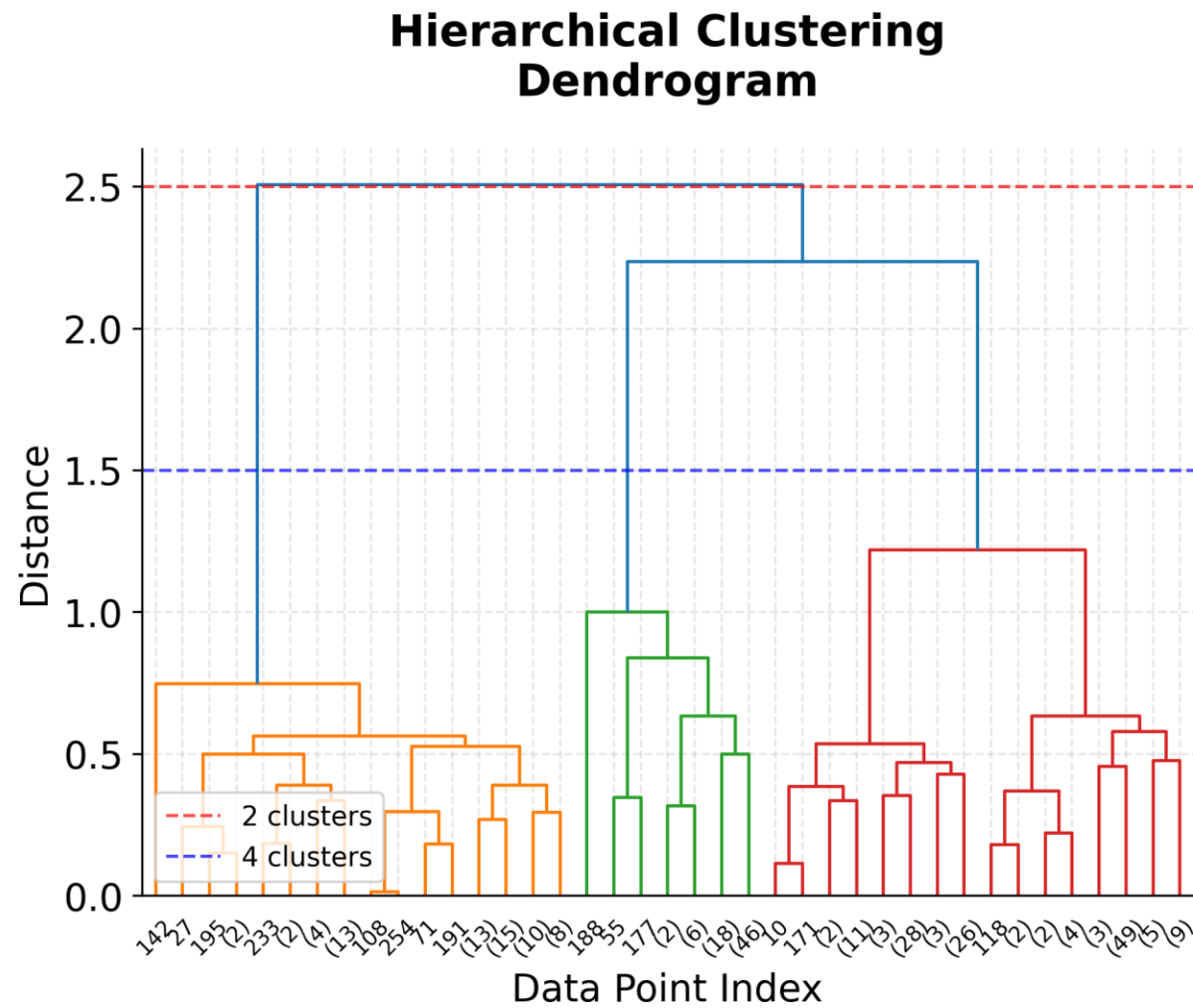
Centroid Linkage



Dendrogramme et code

- Le dendrogramme permet de visualiser les fusions et de choisir a posteriori le nombre de clusters en « coupant » l'arbre à une certaine hauteur.
- `from sklearn.cluster import AgglomerativeClustering`
- `from scipy.cluster.hierarchy import dendrogram, linkage`
- `model = AgglomerativeClustering(n_clusters=3, linkage='ward')`
- `labels = model.fit_predict(X)`
- `Z = linkage(X, method='ward') # pour afficher le dendrogramme`
- `dendrogram(Z)`

Dendrogramme et code



Clustering hiérarchique

Quand utiliser le clustering hiérarchique ?

- 1. On ne connaît pas K à l'avance** On lance l'algorithme une seule fois, puis on choisit le nombre de clusters en regardant le dendrogramme. Pas besoin de tester plusieurs valeurs comme avec K-Means.
- 2. On veut une hiérarchie, pas juste des groupes** Le dendrogramme montre comment les groupes s'emboîtent : sous-groupes, super-groupes, proximités entre clusters. Idéal pour construire une taxonomie.
- 3. Le dataset est petit ou moyen** Méthode lente sur de gros volumes (au-delà de quelques milliers de points). Adaptée à des jeux de données propres et limités.
- 4. On veut un résultat reproductible** Contrairement à K-Means, le clustering hiérarchique est déterministe : même dataset, même résultat à chaque exécution.
- 5. On travaille avec des distances spéciales** Accepte toutes les distances (cosinus, Manhattan, Jaccard, édition de chaînes), pas seulement la distance euclidienne.

Clustering hiérarchique

Cas d'usage typiques

- **Biologie** : classification d'espèces, regroupement de gènes
- **Médecine** : classification de tumeurs, groupes de patients
- **Marketing** : segmentation client avec sous-segments imbriqués
- **NLP** : regroupement de documents avec hiérarchie de thèmes
- **Sciences sociales** : classification de pays selon indicateurs socio-économiques
- **Linguistique** : familles de langues

Clustering hiérarchique

Les cas où il ne faut PAS l'utiliser

- Gros volumes ($> 10\,000$ points) : trop lent, préférer K-Means ou MiniBatchKMeans.
- Données avec beaucoup de bruit ou d'outliers : le hiérarchique va créer des "branches parasites" pour chaque outlier. Préférer DBSCAN qui les identifie comme bruit.
- Quand on connaît K à l'avance et qu'on veut juste partitionner vite : K-Means est plus efficace.
- Quand les clusters ont des formes non sphériques complexes (cercles, lunes) : ni K-Means ni Ward ne marchent bien. Préférer DBSCAN.

DBSCAN : principe

- **K-Means et hiérarchique** raisonnent en **distances** : on regroupe les points proches d'un centre ou les uns des autres.
- **DBSCAN** change de logique : il regroupe les points par zones de forte **densité**, séparées par des zones vides.
- Conséquences :
 - Pas besoin de fixer K à l'avance.
 - Détecte les clusters de forme quelconque (lunes, cercles, spirales), là où K-Means impose des groupes sphériques.
 - Identifie naturellement les outliers comme du bruit, sans qu'on ait à les filtrer en amont.
- Image mentale : un cluster = une "île" de points serrés. Le bruit = les points isolés dans la mer entre les îles.

DBSCAN : les deux paramètres eps et min_samples

- Deux paramètres : eps (rayon de voisinage) et min_samples (nombre minimal de points pour former un cluster).

1- eps (ϵ) : rayon de voisinage autour de chaque point. Définit ce que veut dire "proche".

2- min_samples : nombre minimal de points (le point lui-même inclus) qu'il faut trouver dans le rayon eps pour considérer la zone comme dense.

- Ensemble, ils définissent la densité minimale acceptée pour former un cluster.

Choisir eps avec le k-distance plot :

- Pour chaque point, on calcule la distance à son k-ème voisin ($k = \text{min_samples}$).
- On trie ces distances par ordre croissant et on les trace.
- L'eps optimal correspond au coude de la courbe (changement de pente net).

Règle simple pour min_samples : commencer par $\text{min_samples} = 2 \times \text{nombre de dimensions}$.

Voir reference Ester et al. (1996)

DBSCAN : les trois types de points

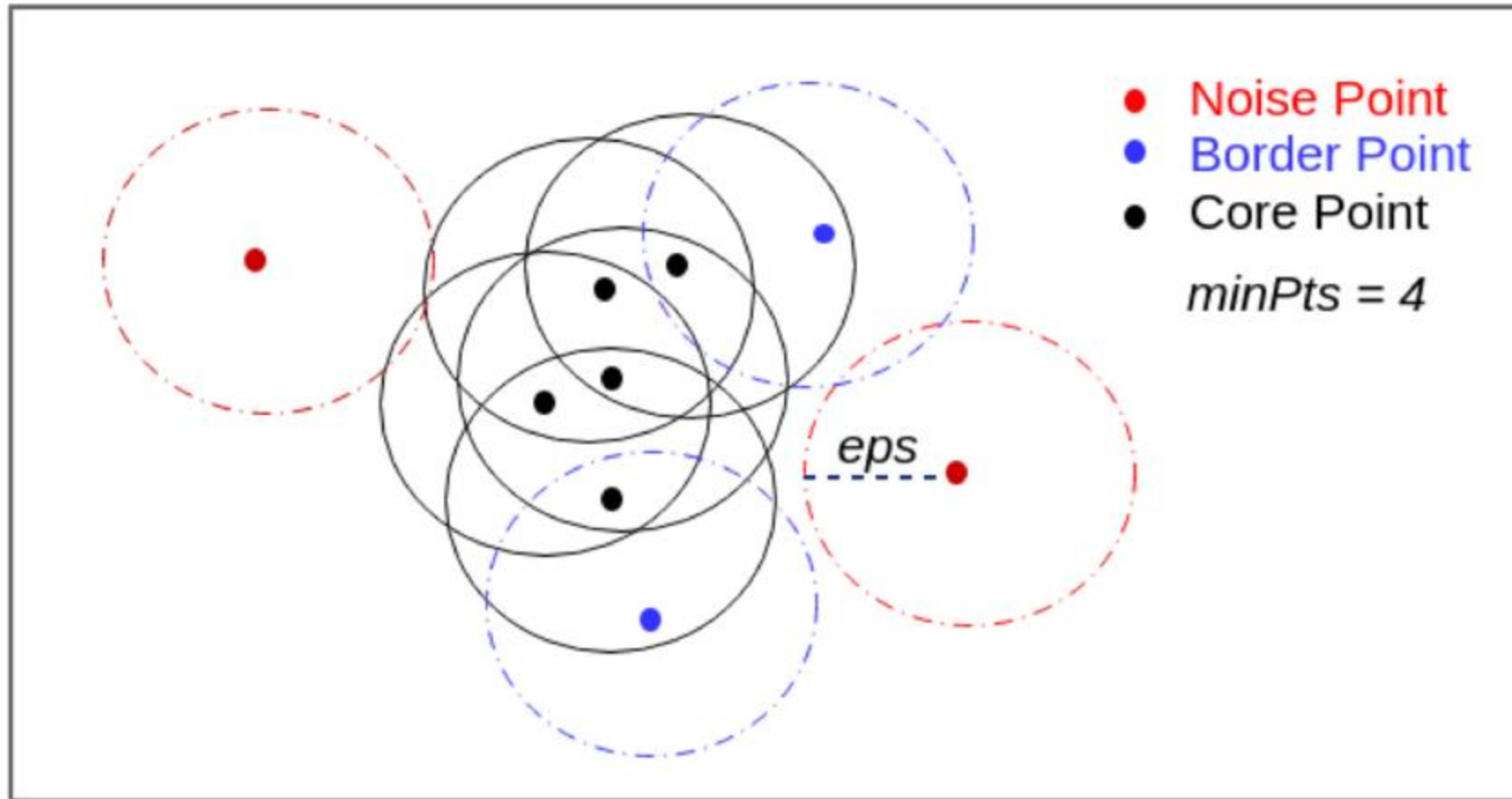
- **Point cœur** : a au moins min_samples voisins dans son rayon eps . C'est lui qui fonde un cluster.
- **Point frontière** : a moins de min_samples voisins, mais tombe dans le rayon eps d'un point cœur. Il est rattaché au cluster, sans pouvoir l'étendre.
- **Point bruit** : n'est ni cœur, ni frontière.

Image mentale :

- Cœur = habitant du centre-ville (entouré de voisins).
- Frontière = habitant de banlieue (peu de voisins, mais proche du centre-ville).
- Bruit = ermite isolé en pleine campagne.

Info : seuls les points cœur peuvent propager un cluster ; les points frontière le rejoignent mais ne le font pas grandir.

DBSCAN : les trois types de points



DBSCAN : code avec scikit-learn

- `from sklearn.cluster import DBSCAN`
- `model = DBSCAN(eps=0.3, min_samples=5)`
- `labels = model.fit_predict(X)`
- Info : les points étiquetés -1 sont considérés comme du bruit.
- Exemple : DBSCAN trouve facilement des clusters de forme « lune » ou « cercle » là où K-Means échoue.
- Exercice : Wholesale Customers (Segmentation client avec quelques gros clients atypiques).

Analyse en composantes principales (PCA) :

- La PCA est une technique de réduction de dimension.
- Objectif : représenter les données dans un espace plus petit tout en conservant le maximum de variance (d'information).
- Utile pour visualiser des données en 2D ou 3D, réduire le bruit et préparer un clustering.
- Les nouvelles variables (composantes principales) sont des combinaisons linéaires des variables d'origine.

PCA : pourquoi réduire la dimension ?

- En ML, on travaille souvent avec beaucoup de variables : 100, 1000, parfois plus. Trois problèmes apparaissent :
 - 1- Malédiction de la dimension : plus on ajoute de features, plus l'espace devient "vide", plus les algorithmes de ML deviennent instables (KNN, K-Means, DBSCAN...).
 - 2- Visualisation impossible : on ne sait pas dessiner un nuage de points en 50 dimensions. On a besoin d'une projection en 2D ou 3D pour voir la structure des données.
 - 3- Bruit dans les données : beaucoup de variables sont redondantes ou bruitées. Les "vraies" informations tiennent souvent dans un nombre réduit de directions.
- L'idée centrale : trouver les axes qui maximisent la variance des données. La variance = la quantité d'information. Ces axes s'appellent les composantes principales.
- Image mentale : un nuage de points incliné dans le plan. Le premier axe principal suit la direction de plus grande dispersion. Le deuxième est perpendiculaire au premier et capture le reste. On projette les points sur ces nouveaux axes, et on garde l'essentiel avec moins de dimensions.

PCA : en pratique

- Mathématiquement, PCA décompose la matrice de covariance des données en valeurs propres et vecteurs propres. Sans rentrer dans le détail mathématique, scikit-learn s'en occupe.

- Code :

```
from sklearn.decomposition import PCA
```

```
pca = PCA(n_components=2)
```

```
X_2d = pca.fit_transform(X_scaled)
```

Info : Toujours standardiser avant PCA. Sinon les variables à grande échelle dominant artificiellement les composantes.

PCA : code et visualisation

- L'outil clé : `explained_variance_ratio_`. Il donne le pourcentage d'information capturé par chaque composante.

Code :

```
print(pca.explained_variance_ratio_)
```

```
# → [0.72, 0.23] → la PC1 capture 72%, la PC2 capture 23%
```

```
# → total 95% : on a conservé 95% de l'information en 2D
```

- Règle pratique : choisir le nombre de composantes pour conserver **80-95%** de la variance. Au-delà, on garde du bruit, en dessous, on perd de l'information utile.

Info : PCA est non supervisée (n'utilise pas les labels), linéaire (n'attrape pas les structures courbes), et déterministe (toujours le même résultat).

Exercice

- Partie 1 : appliquer un clustering hiérarchique sur un dataset d'indicateurs socio-économiques de pays (GDP, espérance de vie, etc.). Afficher le dendrogramme et interpréter les groupes obtenus.
- Partie 2 : générer des données avec `make_moons`. Comparer K-Means, GMM et DBSCAN.
- Partie 3 : sur le dataset Digits (chiffres manuscrits 8x8), réduire à 2 dimensions avec PCA, visualiser et appliquer K-Means.

Web Scrapping et APIs pour le Machine Learning et Deep Learning

Web Scrapping / API : Pourquoi

En cours : on travaille sur des datasets propres déjà prêtes. Dans la vraie vie : les données n'arrivent jamais comme ça.

Il faut souvent aller les chercher soi-même :

- Prix de produits e-commerce
- Cours de la bourse en temps réel
- Données météo
- Articles de presse, posts de réseaux sociaux
- Statistiques publiques (gouv, banque mondiale...)

Web Scrapping / API

DEUX GRANDES APPROCHES

Web Scraping : Télécharger une page HTML et en extraire le contenu (quand pas d'API disponible)

API (Application Programming Interface) : Endpoint renvoie du JSON structuré

Si une API existe → utiliser l'API.

Web Scrapping / API

Avant de scraper, vérifier :

1. Le fichier robots.txt du site

Ex : <https://example.com/robots.txt>

→ Indique ce qui est autorisé

2. Les Conditions d'Utilisation (CGU/ToS)

3. Le RGPD si données personnelles

4. NE PAS surcharger le serveur

→ Ajouter `time.sleep()` entre les requêtes

- LinkedIn, Instagram : scraping interdit et surveillé
- Wikipedia, données gouvernementales : généralement possible
- En cas de doute → s'abstenir

Web Scrapping / API

Une page web = un arbre de balises imbriquées

```
<html>
```

```
<body>
```

```
<h1>Titre de la page</h1>
```

```
<div class="produit">
```

```
<span class="prix">19.99 €</span>
```

```
<span class="nom">Casque audio</span>
```

```
</div>
```

```
</body>
```

```
</html>
```

Scraper = naviguer dans cet arbre pour extraire ce qu'on veut.

ÉLÉMENTS CLÉS :

- Balises : <div>, , <a>, <p>, <h1>...
- Attributs : class="...", id="...", href="..."
- Texte : contenu entre les balises